
1. How CRC Works (Simple Idea)

1. Sender has **data bits**.
 2. A **generator polynomial (key)** is chosen.
 3. Sender performs **modulo-2 division**.
 4. The **remainder** is appended to the data.
 5. Receiver divides the received message by the same key.
 6. If **remainder = 0** → **No error**
If **remainder ≠ 0** → **Error detected**
-

2. CRC Function (Used by both client and server)

```
def xor(a, b):  
    result = ""  
    for i in range(1, len(b)):  
        if a[i] == b[i]:  
            result += "0"  
        else:  
            result += "1"  
    return result  
  
def mod2div(dividend, divisor):  
    pick = len(divisor)  
    tmp = dividend[0:pick]  
  
    while pick < len(dividend):  
        if tmp[0] == '1':
```

```

        tmp = xor(divisor, tmp) + dividend[pick]
    else:
        tmp = xor('0'*pick, tmp) + dividend[pick]
    pick += 1

    if tmp[0] == '1':
        tmp = xor(divisor, tmp)
    else:
        tmp = xor('0'*pick, tmp)

    return tmp

def encode_data(data, key):
    l_key = len(key)
    appended_data = data + '0'*(l_key-1)
    remainder = mod2div(appended_data, key)
    codeword = data + remainder
    return codeword

```

What this does

- `xor()` → performs XOR operation
- `mod2div()` → performs CRC division
- `encode_data()` → adds CRC bits to data

3. Server Program

```

import socket

def xor(a, b):
    result = ""
    for i in range(1, len(b)):
        if a[i] == b[i]:

```

```

        result += "0"
    else:
        result += "1"
    return result

def mod2div(dividend, divisor):
    pick = len(divisor)
    tmp = dividend[0:pick]

    while pick < len(dividend):
        if tmp[0] == '1':
            tmp = xor(divisor, tmp) + dividend[pick]
        else:
            tmp = xor('0'*pick, tmp) + dividend[pick]
        pick += 1

    if tmp[0] == '1':
        tmp = xor(divisor, tmp)
    else:
        tmp = xor('0'*pick, tmp)

    return tmp

key = "1101"

s = socket.socket()
s.bind(("localhost", 12345))
s.listen(1)

print("Server waiting...")

conn, addr = s.accept()

data = conn.recv(1024).decode()

```

```
print("Received data:", data)

remainder = mod2div(data, key)

if "1" in remainder:
    print("Error detected in received data")
else:
    print("No error detected")

conn.close()
```

4. Client Program

```
import socket

def xor(a, b):
    result = ""
    for i in range(1, len(b)):
        if a[i] == b[i]:
            result += "0"
        else:
            result += "1"
    return result

def mod2div(dividend, divisor):
    pick = len(divisor)
    tmp = dividend[0:pick]

    while pick < len(dividend):
        if tmp[0] == '1':
            tmp = xor(divisor, tmp) + dividend[pick]
        else:
```

```

        tmp = xor('0'*pick, tmp) + dividend[pick]
        pick += 1

    if tmp[0] == '1':
        tmp = xor(divisor, tmp)
    else:
        tmp = xor('0'*pick, tmp)

    return tmp

def encode_data(data, key):
    l_key = len(key)
    appended_data = data + '0'*(l_key-1)
    remainder = mod2div(appended_data, key)
    codeword = data + remainder
    return codeword

key = "1101"
data = input("Enter data bits: ")

codeword = encode_data(data, key)

print("Data with CRC:", codeword)

s = socket.socket()
s.connect(("localhost", 12345))

s.send(codeword.encode())

s.close()

```

5. How to Run It

Step 1

Run the **server first**

```
python server.py
```

Step 2

Run the **client**

```
python client.py
```

Step 3

Enter binary data like:

```
101100
```

Client sends:

```
101100 + CRC bits
```

Server checks the CRC.

6. Example Output

Client:

```
Enter data bits: 101100  
Data with CRC: 101100111
```

Server:

```
Server waiting...  
Received data: 101100111
```

No error detected

7. Simple Flow Diagram

```
Client
|
| 1. Input data
| 2. Calculate CRC
| 3. Send data+CRC
v
Server
|
| 4. Recalculate CRC
| 5. If remainder=0 → OK
| 6. Else → Error
```

✓ If you want, I can also show you:

- **A much shorter CRC client-server program (exam friendly – ~30 lines)**
- **How to intentionally introduce an error to test CRC detection**
- **A diagram explanation that makes CRC extremely easy to remember for exams.**

```
import socket

def xor(a, b):
    result = ""
    for i in range(1, len(b)):
        if a[i] == b[i]:
            result += "0"
        else:
            result += "1"
    return result
```

```

def mod2div(dividend, divisor):
    pick = len(divisor)
    tmp = dividend[0:pick]

    while pick < len(dividend):
        if tmp[0] == '1':
            tmp = xor(divisor, tmp) + dividend[pick]
        else:
            tmp = xor('0'*pick, tmp) + dividend[pick]
        pick += 1

    if tmp[0] == '1':
        tmp = xor(divisor, tmp)
    else:
        tmp = xor('0'*pick, tmp)

    return tmp

def encode_data(data, key):
    l_key = len(key)
    appended_data = data + '0'*(l_key-1)
    remainder = mod2div(appended_data, key)
    codeword = data + remainder
    return codeword

def introduce_error(data):
    pos = int(input("Enter bit position to flip (starting from 0): "))

    data = list(data)
    if data[pos] == '0':
        data[pos] = '1'
    else:
        data[pos] = '0'

```



```

        return ''.join(data)

key = "1101"

data = input("Enter data bits: ")

encoded_data = encode_data(data, key)

print("Encoded data (before error):", encoded_data)

choice = input("Do you want to introduce error? (y/n): ")

if choice.lower() == 'y':
    encoded_data = introduce_error(encoded_data)
    print("Data after introducing error:", encoded_data)

s = socket.socket()
s.connect(("localhost", 12345))

s.send(encoded_data.encode())

print("Data sent to server")

s.close()

```